Javascript

Dominando o JavaScript Assíncrono

Prof. Maromo

Aula 07

2025

Agenda

- O que é Código Assíncrono?
- Callbacks e o Problema do "Callback Hell"
- A Solução Moderna: Promises
- A Sintaxe Mais Limpa: Async/Await
- Requisições de Rede com a Fetch API
- Laboratório Prático: Buscando Dados da API do GitHub

O que é Código Assíncrono?

- Por padrão, o JavaScript executa uma operação por vez, em sequência.
- Se uma tarefa demorar muito (como fazer o download de uma imagem ou buscar dados de um servidor), ela "bloqueia" todas as outras.
- O JavaScript assíncrono permite que operações demoradas sejam executadas em segundo plano, sem travar a aplicação.

Callbacks e o Problema do "Callback Hell"

- Callbacks são a forma mais fundamental de lidar com assincronicidade.
- Um callback é **simplesmente uma função que é passada como argumento para outra função**, para ser executada ("called back") **quando a operação principal terminar**.

```javascript
//Exemplo 01- Exemplo de callback simples

function obterDados(callback) {
    // Simula uma operação que leva 2 segundos
    setTimeout(function() {
        const dados = { nome: 'Maromo', id: 71 };
        // Executa o callback com os dados quando estiver pronto
        callback(dados);
    }, 2000);
}

function mostrarDados(dados) {
    console.log(dados);
}

// Passamos 'mostrarDados' como um callback
obterDados(mostrarDados);
```

Exemplo Callback

Problema

- O principal problema surge quando múltiplas operações assíncronas dependem umas das outras.
- Isso leva a um aninhamento excessivo de callbacks, conhecido como **"Callback Hell"**, que torna o código difícil de ler e dar manutenção.
- Veja a ilustração do Callback Hell:

Ilustrando Callback Hell

```javascript
// Exemplo 02 - Ilustração do "Callback Hell"
// Funções que simulam operações assíncronas (ex: chamada de API,
// acesso a banco de dados)
function primeiraFuncao(callback) {
    console.log("Iniciando primeira função...");
    setTimeout(function() {
        const resultado1 = "Resultado 1";
        console.log("Primeira função concluída.");
        callback(resultado1);
    }, 1000); // Leva 1 segundo
}
```

```javascript
function segundaFuncao(dado, callback) {
    console.log("Iniciando segunda função com:", dado);
    setTimeout(function() {
        const resultado2 = dado + ", Resultado 2";
        console.log("Segunda função concluída.");
        callback(resultado2);
    }, 1000); // Leva 1 segundo
}
```

```javascript
function terceiraFuncao(dado, callback) {
    console.log("Iniciando terceira função com:", dado);
    setTimeout(function() {
        const resultado3 = dado + ", Resultado 3";
        console.log("Terceira função concluída.");
        callback(resultado3);
    }, 1000); // Leva 1 segundo
}

```

```javascript
// Este aninhamento é o "Callback Hell".
// Cada função só é chamada após a anterior ser concluída.
primeiraFuncao(function(resultado1) {
    segundaFuncao(resultado1, function(resultado2) {
        terceiraFuncao(resultado2, function(resultado3) {
            console.log('\nResultado final de todas as operações:');
            console.log(resultado3);
        });
    });
});
```

A Solução Moderna: Promises

- Promessas (Promises) foram criadas para resolver o "Callback Hell".
- Uma Promise é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona.
- Uma promessa pode estar em um de três estados:
  - **Pendente (Pending):** Estado inicial, a operação ainda não foi concluída.
  - **Resolvida (Fulfilled):** A operação foi concluída com sucesso.
  - **Rejeitada (Rejected):** A operação falhou.

```javascript
// Exemplo 03 - Uso de Promises para evitar "Callback Hell"
// Função que retorna uma Promise simul
function obterDadosComPromise() {
    return new Promise(function(resolve, reject) {
        setTimeout(function() {
            const dados = 'Dados obtidos com sucesso!';
            if (dados) {
                resolve(dados); // A promessa foi resolvida
            } else {
                reject('Erro ao obter dados'); // A promessa foi rejeitada
            }
        }, 2000);
    });
}
// Usando a função com Promises
obterDadosComPromise()
    .then(function(resultado) {
        console.log(resultado); // Será executado em caso de sucesso
    })
    .catch(function(erro) {
        console.error(erro); // Será executado em caso de falha
    });
```

A Sintaxe Mais Limpa: Async/Await

- Async/Await é uma sintaxe mais moderna que torna o código assíncrono mais fácil de ler, fazendo-o parecer síncrono. Por baixo dos panos, ele ainda utiliza Promises.
  - **async:** É usada para declarar que uma função retornará uma promessa.
  - **await:** Só pode ser usada dentro de uma função async. Ela pausa a execução da função até que uma Promise seja resolvida.
  - Erros são tratados com um bloco **try/catch tradicional**.

```javascript
// Reutilizando a função 'obterDadosComPromise' do exemplo anterior
function obterDadosComPromise() {
    return new Promise(function(resolve, reject) {
        setTimeout(function() {
            const dados = 'Dados obtidos com sucesso!';
            if (dados) {
                resolve(dados); // A promessa foi resolvida
            } else {
                reject('Erro ao obter dados'); // A promessa foi rejeitada
            }
        }, 2000);
    });
}
async function mostrarDados() {
    try {
        console.log('Esperando os dados...');
        const dados = await obterDadosComPromise(); // Pausa a execução aqui
        console.log(dados); // Esta linha só executa após a promessa ser resolvida
    } catch (erro) {
        console.error('Ocorreu um erro:', erro);
    }
}

mostrarDados();
```

Requisições de Rede com a Fetch API

- A Fetch API é a ferramenta nativa e moderna do JavaScript para fazer requisições HTTP (buscar dados de um servidor ou API).
- Ela substitui a abordagem antiga com Ajax (XMLHttpRequest) e é baseada em Promises, o que a torna perfeita para usar com **.then()** ou **async/await**.

```
1   fetch('https://api.github.com/users/maromo71')
2       .then(function(response) {
3           // O primeiro .then trata a resposta HTTP
4           return response.json(); // Converte a resposta para o formato JSON
5       })
6       .then(function(data) {
7           // O segundo .then lida com os dados já convertidos
8           console.log(data);
9       })
10      .catch(function(error) {
11          console.error('Falha na requisição:', error);
12      });
```

{
  login: 'maromo71',
  id: 9035218,
  node_id: 'MDQ6VXNlcjkwMzUyMTg=',
  avatar_url: 'https://avatars.githubusercontent.com/u/9035218?v=4',
  gravatar_id: '',
  url: 'https://api.github.com/users/maromo71',
  html_url: 'https://github.com/maromo71',
  followers_url: 'https://api.github.com/users/maromo71/followers',
  following_url: 'https://api.github.com/users/maromo71/following{/other_user}',
  gists_url: 'https://api.github.com/users/maromo71/gists{/gist_id}',
  starred_url: 'https://api.github.com/users/maromo71/starred{/owner}{/repo}',
  subscriptions_url: 'https://api.github.com/users/maromo71/subscriptions',
  organizations_url: 'https://api.github.com/users/maromo71/orgs',
  repos_url: 'https://api.github.com/users/maromo71/repos',
  events_url: 'https://api.github.com/users/maromo71/events{/privacy}',
  received_events_url: 'https://api.github.com/users/maromo71/received_events',
  type: 'User',
  user_view_type: 'public',
  site_admin: false,
  name: 'Maromo',
  company: 'Fatec Mogi Mirim',
  blog: 'maromo.pro.br',
  location: 'Mogi Mirim',
  email: null,
  hireable: null,
  bio: 'Graduado em Processamento de Dados (1995)\r\n' +
    'Especialista em Administração de Sistemas de Informação (2005)\r\n' +
    'Mestre em Educação (2010)',
  twitter_username: null,
  public_repos: 199,
  public_gists: 2,
  followers: 438,
  following: 10,
  created_at: '2014-10-06T11:36:57Z',
  updated_at: '2025-09-19T23:51:11Z'
}

Laboratório

- Buscando Dados da API do GitHub

- **Objetivo:** Criar uma página que busca os dados de um usuário do GitHub e os exibe em uma tabela.

Arquivo: index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Exibindo Dados da API</title>
</head>
<body>
    <h1>Dados do Usuário do GitHub</h1>
    <table id="dados-table" border="1">
        <thead>
            <tr>
                <th>Nome</th>
                <th>Repositórios Públicos</th>
                <th>Seguidores</th>
            </tr>
        </thead>
        <tbody>
            </tbody>
    </table>
    <script src="script.js"></script>
</body>
</html>
```

Arquivo: script.js

```javascript
// Função que busca os dados na API
async function obterDadosUsuario(nomeUsuario) {
    const url = `https://api.github.com/users/${nomeUsuario}`;
    try {
        const response = await fetch(url);
        if (!response.ok) {
            // Se a resposta não for bem-sucedida (ex: erro 404), lança um erro
            throw new Error('Erro ao obter dados do usuário');
        }
        const data = await response.json();
        return data;
    } catch (error) {
        console.error('Falha na busca:', error);
        // Retorna null ou lança o erro para quem chamou a função decidir como tratar
        return null;
    }
}
```

Arquivo: script.js

```javascript
// Função que exibe os dados na tabela HTML
function exibirDadosUsuario(usuario) {
    const tableBody = document.querySelector('#dados-table tbody');

    // Limpa a mensagem de "Carregando..."
    tableBody.innerHTML = '';

    const row = document.createElement('tr');

    const nomeCell = document.createElement('td');
    const reposCell = document.createElement('td');
    const seguidoresCell = document.createElement('td');

    nomeCell.textContent = usuario.name;
    reposCell.textContent = usuario.public_repos;
    seguidoresCell.textContent = usuario.followers;

    row.appendChild(nomeCell);
    row.appendChild(reposCell);
    row.appendChild(seguidoresCell);

    tableBody.appendChild(row);
}
```

Finalizando: arquivo script.js

```
// Função principal que orquestra a execução
async function iniciar() {
    const tableBody = document.querySelector('#dados-table tbody');
    tableBody.innerHTML = '<tr><td colspan="3">Carregando...</td></tr>';

    const usuario = await obterDadosUsuario('maromo71');

    if (usuario) {
        exibirDadosUsuario(usuario);
    } else {
        tableBody.innerHTML = '<tr><td colspan="3">Não foi possível carregar os dados.</td></tr>';
    }
}

// Inicia a execução do nosso script
iniciar();
```

Resumo

- **Marco Fundamental:** é crucial no aprendizado de JavaScript, pois introduz o conceito de assincronicidade.
- **Aplicações Modernas:** O conteúdo é essencial para criar aplicações web modernas e que respondem bem à interação do usuário.
- **Jornada Evolutiva:** O material guia o aluno pela evolução das técnicas assíncronas, começando com Callbacks e suas limitações ("Callback Hell").
- **Soluções Modernas:** Apresenta as soluções mais atuais e legíveis, como as Promises e a sintaxe Async/Await.

Fim

Maromo